

Assignment: Exploring Image Segmentation and Super-Resolution with CNNs

Overview

In the lab demo, you explored two important computer vision problems:

Task	Goal
Image Segmentation	Predict a class label for every pixel in an image
Image Super-Resolution	Recover a high-resolution image from a low-resolution one

Both tasks are examples of **dense prediction problems** — the model must produce an output at the pixel level rather than predicting a single label for the whole image.

In this assignment you will:

- Train a **baseline segmentation model**
- Improve it using **skip connections (U-Net)**
- Explore **refined skip connections inspired by U-Net++**
- Implement **super-resolution networks (SRCNN)**
- Understand **residual learning (VDSR idea)**
- Perform **small ablation studies** to understand architectural design

The goal is not only to run models, but to understand **why certain architectural ideas improve performance**.

Dataset

For this assignment you will use the **Oxford-IIIT Pet Dataset**.

The Oxford-IIIT Pet Dataset contains photographs of cats and dogs belonging to **37 breeds**, along with **pixel-level segmentation masks** that identify the location of the pet in each image.

The dataset is widely used in computer vision tutorials and is conveniently available directly through `torchvision.datasets` in PyTorch.

We will use **the same dataset for both tasks in this assignment**.

Using Oxford-IIIT Pet for Image Segmentation

Each image in the dataset has a corresponding **segmentation mask**.

The segmentation masks label each pixel as one of three categories:

- **Pet (foreground)**
- **Background**
- **Border**

For simplicity in this assignment, you may convert the mask into a **binary segmentation task**:

- **Foreground:** pet pixels
- **Background:** non-pet pixels

This simplifies the learning problem and allows you to train lightweight models such as the baseline CNN and Mini U-Net.

For the segmentation task:

- **Input:** RGB image of a pet
- **Target:** binary segmentation mask

The model learns to predict **which pixels belong to the pet**.

Using Oxford-IIIT Pet for Image Super-Resolution

The same images from the Oxford-IIIT Pet dataset will be used to generate **low-resolution (LR) and high-resolution (HR) image pairs** for the super-resolution task.

To create training pairs:

1. Start with the **original image** (this will be the **high-resolution image**).
2. Downscale the image using **bicubic interpolation** (for example, by a factor of $\times 2$).
3. Upscale the low-resolution image back to the original size using **bicubic interpolation**.
4. Use this upscaled image as the **input** to the super-resolution model.

The model learns to reconstruct the **original high-resolution image** from the degraded version.

Summary of Dataset Usage

Task	Input	Target
Image Segmentation	Pet image	Binary segmentation mask
Super-Resolution	Bicubic-upscaled LR image	Original HR image

Using the same dataset for both tasks allows you to focus on understanding the **model architectures and training behavior**, rather than dealing with multiple datasets.

Part 1 — Baseline Segmentation Model

In the demo notebook, you trained a simple **encoder-decoder CNN** for segmentation.

The model predicts:

$$\text{Image} \rightarrow \text{CNN} \rightarrow \text{Segmentation Mask}$$

Each pixel in the output mask represents the **probability that the pixel belongs to the object**.

Segmentation Evaluation Metric — IoU

Segmentation models are typically evaluated using **Intersection over Union (IoU)**.

If:

- P = predicted mask
- G = ground truth mask

then

$$IoU = \frac{|P \cap G|}{|P \cup G|}$$

Interpretation

IoU Value	Meaning
1.0	Perfect overlap
0.7 – 0.9	Good segmentation
< 0.5	Poor segmentation

IoU measures how well the predicted region overlaps the ground truth region.

Task 1 — Train the Baseline Model

Run the baseline segmentation model provided in the demo.

Record:

- Training loss
- Validation **IoU**
- Example predicted masks

Questions

1. What kinds of errors do you observe in the predicted masks?
2. Are object boundaries predicted accurately?
3. Does the model struggle with **thin structures (ears, tails, whiskers)**?

Explain your observations briefly.

Part 2 — Improving Segmentation with Skip Connections (U-Net)

The baseline model downsamples images using pooling layers. This causes **loss of spatial details** such as edges and fine boundaries.

One of the most influential segmentation architectures is **U-Net**.

Core Idea of U-Net

U-Net has two parts:

Encoder

- Extracts features
- Reduces spatial resolution

Decoder

- Reconstructs segmentation masks
- Upsamples features

U-Net introduces **skip connections**:

Encoder feature $\rightarrow$ concatenate $\rightarrow$ Decoder feature

These connections allow the decoder to reuse **high-resolution spatial information** from earlier layers.

Task 2 — Implement a Mini U-Net

Modify the baseline network to include skip connections.

Architecture requirements:

- Encoder: at least **2 convolution blocks**
- Decoder: at least **2 upsampling blocks**
- Use **ConvTranspose2d** for upsampling
- Use `torch.cat()` for concatenating encoder features

Train the model using the same dataset.

Questions

1. Compare IoU between:
 - Baseline CNN
 - Mini U-Net
2. Do skip connections improve boundary predictions?
3. Why might combining **low-level spatial features** and **high-level semantic features** help segmentation?

Ablation Study 1 — Model Depth

Modify your Mini U-Net:

- Add one additional encoder layer
- Add one additional decoder layer

Compare:

Model	IoU
Mini U-Net (2 layers)	
Deeper Mini U-Net (3 layers)	

Questions

1. Did increasing depth improve IoU?
2. Was there a noticeable increase in training time?
3. Did deeper models produce sharper masks?

Part 3 — Understanding U-Net++

Encoder and decoder features are very different in nature:

Encoder	Decoder
Low-level textures	High-level semantic features

Directly concatenating them creates a **semantic gap**.

U-Net++ reduces this gap by refining encoder features before merging them with decoder features.

Task 4 — Implement a Simplified U-Net++ Idea

Before concatenating the encoder feature with the decoder feature, pass the encoder feature through a refinement block:

$$\text{Conv}(3 \times 3) \rightarrow \text{ReLU}$$

Then concatenate.

Train this model with the same pipeline.

Questions

Model	IoU
Baseline CNN	
Mini U-Net	
Mini U-Net++	

1. Does feature refinement improve segmentation?
2. Why might refining encoder features help reduce the semantic gap?

Part 4 — Image Super-Resolution

Image super-resolution reconstructs a high-resolution image from a low-resolution input.

Super-Resolution Metric — PSNR

Super-resolution models are commonly evaluated using **Peak Signal-to-Noise Ratio (PSNR)**.

PSNR is derived from **Mean Squared Error (MSE)**.

Higher PSNR indicates better reconstruction quality.

PSNR	Interpretation
20–25 dB	Low quality
25–30 dB	Reasonable reconstruction
30+ dB	High quality

Part 5 — SRCNN

SRCNN learns a mapping:

Upscaled LR Image $\rightarrow$ HR Image

The original architecture used kernels:

$$9 \times 9 \rightarrow 5 \times 5 \rightarrow 5 \times 5$$

Task 5 — Kernel Size Experiment

Train two versions:

Model A

$$9 \times 9 \rightarrow 5 \times 5 \rightarrow 5 \times 5$$

Model B

$$5 \times 5 \rightarrow 3 \times 3 \rightarrow 3 \times 3$$

Record training loss and validation PSNR.

Questions

1. Which configuration produced higher PSNR?
2. Why might larger kernels help capture image structures?

Task 6 — Learning Rate Experiment

Train SRCNN with two learning rates:

10^{-3}	and	10^{-4}
Learning Rate	PSNR	
10^{-3}		
10^{-4}		

Part 6 — Residual Learning

Residual learning predicts:

$$\text{Residual} = HR - \text{Bicubic}(LR)$$

Final reconstruction:

$$SR = \text{Bicubic}(LR) + \text{Residual}$$

Task 7 — Implement Residual Super-Resolution

Modify SRCNN so that:

```
residual = model(lr_img)
sr_image = lr_img + residual
```

Compute PSNR.

Questions

1. Compare PSNR between standard SRCNN and residual SRCNN.
2. Did the residual model converge faster?
3. Why is predicting only missing details easier?

Part 7 — Qualitative Comparison

Select three validation images and display:

- Bicubic upscaling
- SRCNN output
- Residual SRCNN output
- Ground truth HR image

Questions

1. Which textures are hardest to reconstruct?
2. Do models recover sharp edges or smooth regions better?
3. Does higher PSNR always correspond to better visual quality?

Submission Requirements

Your submission should include:

- Implementation of all models
- Tables comparing **IoU and PSNR**
- Visualization outputs for segmentation and super-resolution
- Short written answers to conceptual questions

Focus on interpreting your results, not just training models.